Bài tập thực hành tuần 1

> Họ và tên: Nguyễn Vạn Phúc Huy
> MSSV: 23110163
> Lớp: 23TTH (Sáng thứ 6)

Bài tập 1.1:

Câu hỏi:

1. Viết chương trình để biểu diễn một số thập phân N sang dạng biểu diễn nhị phân có độ phức tạp thuật toán là $O(\log_2 N)$.
   - Input: Số thập phân N.
   - Output: Dạng biểu diễn nhị phân của N.

```python
def binary(n: int):
    if n < 0:
        raise ValueError("Error: n is negative")
    bin = ''
    while n > 0:
        div = str(n % 2) # convert to string because 01 = 1
        bin += div
        n //= 2
    return bin[::-1]
input = [1, 2, 3, 100, 23110163]
for n in input:
    print(f'binary({n}) = {binary(n)}')
```

Ta kiểm tra điều kiện n < 0 để đảm bảo rằng hàm `binary` chỉ nhận các số nguyên dương. Giúp ngăn việc xử lý các giá trị không hợp lệ và đảm bảo hoạt động đúng.

```python
if n < 0:
    raise ValueError("Error: n is negative")
```

Ta sử dụng một vòng lặp `while` để liên tục chia số `n` cho 2 và lấy phần dư (sử dụng toán tử `%`) để xây dựng chuỗi nhị phân. Phần dư sẽ là 0 hoặc 1, và ta thêm nó vào chuỗi `bin`. Sau đó, ta cập nhật `n` bằng cách chia nguyên cho 2.

```python
bin = ''
while n > 0:
    div = str(n % 2) # convert to string because 01 = 1
    bin += div
    n //= 2
```

Sau while kết thúc, `bin` sẽ chứa các chữ số nhị phân theo thứ tự ngược lại (tức là từ thấp đến cao). Do đó `[::-1]` để đảo ngược chuỗi và trả về kết quả đúng.

```python
return bin[::-1]
```

Bài tập 1.2:

Câu hỏi:

2. Giả sử các em đã xây dựng thuật toán trên. Hãy viết một chương trình để đếm số phép gán và số phép so sánh mà chương trình trên đã dùng để biểu diễn một số thập phân N.

   - Input: binary(N), $N = 100, 200, 300, 400, \ldots, 1000$.
   - Output: $Gan(N), Sosanh(N)$. Vẽ $\log_2 N, Gan(N)$, và $Sosanh(N)$ trên cùng một đồ thị để so sánh.

```python
def binary(n):
    if n < 0:
        raise ValueError("Error: n is negative")
    # skip n < 0 because input is N = 100, 200, ..., 1000

    gan = 0
    soSanh = 0
    bin = ''
    while n:
        soSanh += 1
        div = str(n % 2)
        bin += div
        n //= 2

        gan += 3 # 1 for div, 1 for bin, 1 for n

    soSanh += 1 # Because the last comparison break the while loop
    return bin[::-1], gan, soSanh


N = [100*i for i in range(1, 11)]
assignments, comparisons = [], []
for n in N:
    bin, gan, soSanh = binary(n)
    assignments.append(gan)
    comparisons.append(soSanh)
    print(f'binary({n}) = {bin} with assignments = {gan}, comparisons = {soSanh}')

# Plot for Gan(N) and SoSanh(N) and Log2N
from math import log2
import matplotlib.pyplot as plt

log2_plot = list(map(lambda x: log2(x), N))
plt.plot(log2_plot, label='Log2N')
plt.plot(assignments, label='Gan(N)')
plt.plot(comparisons, label='Sosanh(N)')
plt.xlabel('N')
plt.xticks([i for i in range(10)], N)
plt.legend()
plt.show()
```

Giống như 'binary' phân tích ở trên thì ở đây ta chỉnh lại thì ta thêm các biến đếm

```python
gan = 0
soSanh = 0
```

Trong vòng lặp `while`, ta tăng biến `soSanh` mỗi lần so sánh điều kiện `n` và tăng biến `gan` cho mỗi phép gán (gồm phép gán `div`, phép gán `bin`, và phép gán `n`).

```python
while n:
    soSanh += 1
    div = str(n % 2)
    bin += div
```

```
    n //= 2

    gan += 3 # 1 for div, 1 for bin, 1 for n
```

Sau khi vòng lặp kết thúc, ta cũng tăng `soSanh` thêm 1 để đếm lần so sánh cuối cùng đã làm cho điều kiện `n` trở thành False.

```
soSanh += 1 # Because the last comparison break the while loop
```

Cuối cùng, ta trả về chuỗi nhị phân cùng với số lượng phép gán và so sánh đã thực hiện.

```
return bin[::-1], gan, soSanh
```

Sau đó, ta chạy hàm `binary` cho các giá trị `N` từ 100 đến 1000 và lưu trữ số lượng phép gán và so sánh vào các danh sách `assignments` và `comparisons`. Cuối cùng, ta vẽ đồ thị để so sánh giữa `Gan(N)`, `SoSanh(N)`, và `Log2N`.

```
N = [100*i for i in range(1, 11)]
assignments, comparisons = [], []
for n in N:
    bin, gan, soSanh = binary(n)
    assignments.append(gan)
    comparisons.append(soSanh)
    print(f'binary({n}) = {bin} with assignments = {gan}, comparisons = {soSanh}')
# Plot for Gan(N) and SoSanh(N) and Log2N
from math import log2
import matplotlib.pyplot as plt

log2_plot = list(map(lambda x: log2(x), N))
plt.plot(log2_plot, label='Log2N')
plt.plot(assignments, label='Gan(N)')
plt.plot(comparisons, label='Sosanh(N)')
plt.xlabel('N')
plt.xticks([i for i in range(10)], N)
plt.legend()
plt.show()
```

Note: tạo mảng assignments và comparisons để lưu trữ số lượng phép gán và so sánh tương ứng với mỗi giá trị của N, sau đó sử dụng chúng để vẽ đồ thị.

Bài tập 2:

Bài 2 (Bonus): Cho một mảng A gồm n số tự nhiên nằm $[1 \dots k]$. Hãy thiết kế thuật toán để kiểm tra xem có bao nhiêu phần tử của A nằm trong $[a, b]$ có độ phức tạp là $O(N + k)$ và chứng minh thuật toán của các em đưa ra có độ phức tạp như trên.

- Input: Xét hai trường hợp
  - $N = 10, 20, 30, \dots, 10000$ (k cố định, $k = 100$).
  - $k = 10, 20, 30, \dots$ (N cố định, $N = 20000$).

  Tạo ngẫu nhiên mảng A với $[a, b]$ tương ứng.
- Output: Đếm số phép gán, số phép so sánh và so sánh với $O(N + k)$.

Câu hỏi:

```
from random import randint

def create_ab(k):
    a = randint(1, k)
    b = randint(1, k)
    while a >= b:
        a = randint(1, k)
        b = randint(1, k)
```

```python
    return a, b

def createRandomArray(N, k):
    return [randint(1, k) for _ in range(N)]

def algorithm(A, a, b, k):
    count = [0] * (k + 1)
    cum_sum = [0] * (k + 1)
    assignment = 1
    comparison = 0

    for num in A:
        count[num] += 1
        assignment += 1


    cum_sum[0] = count[0]
    assignment += 2

    for i in range(1, k + 1):
        cum_sum[i] = cum_sum[i - 1] + count[i]
        assignment += 1

    result = cum_sum[b] - cum_sum[a - 1]
    return result, assignment, comparison

# TH1
bigO_list = []
assignments = []
comparisons = []
k = 100
for N in [10*i for i in range(1, 1001)]:
    A = createRandomArray(N, k)
    a, b = create_ab(k)
    result, assignment, comparison = algorithm(A, a, b, k)
    bigO = N + k

    bigO_list.append(bigO)
    assignments.append(assignment)
    comparisons.append(comparison)

plt.figure(figsize=(10, 5))
plt.plot(bigO_list, label='O(N+k)')
plt.plot(assignments, label='Gan(N, k)')
plt.plot(comparisons, label='Sosanh(N, k)')
plt.xlabel('N')
plt.legend()
plt.title('TH 1: k = 100')
plt.show()

# TH2
bigO_list = []
assignments = []
comparisons = []
N = 20000
for k in [10*i for i in range(1, 1001)]:
    A = createRandomArray(N, k)
    a, b = create_ab(k)
    result, assignment, comparison = algorithm(A, a, b, k)
    bigO = N + k
    bigO_list.append(bigO)
    assignments.append(assignment)
    comparisons.append(comparison)

plt.figure(figsize=(10, 5))
```

```
plt.plot(bigO_list, label='O(N+k)')
plt.plot(assignments, label='Gan(N, k)')
plt.plot(comparisons, label='Sosanh(N, k)')
plt.xlabel('k')
plt.legend()
plt.title('TH 2: N = 20000')
plt.show()
```

Để thuật toán đạt O(N + k), ta cần đảm bảo rằng số lượng phép gán và so sánh của thuật toán không vượt quá một hằng số nhân với N + k. Trong thuật toán đã cho, ta có:

- Vòng lặp đầu tiên chạy qua tất cả phần tử của mảng A, thực hiện N phép gán (để update ' `count`) và N phép so sánh (để kiểm tra điều kiện của vòng lặp).

```
for num in A:
    count[num] += 1
    assignment += 1
```

- Vòng lặp thứ hai chạy qua tất cả phần tử của mảng cum_sum, thực hiện k phép gán (để cập nhật mảng cum_sum) và k phép so sánh (để kiểm tra điều kiện của vòng lặp).

```
for i in range(1, k + 1):
    cum_sum[i] = cum_sum[i - 1] + count[i]
    assignment += 1
```

- Để chứng minh rõ hơn thì chúng ta có đồ thị số lượng phép gán và so sánh theo N và k, ta sẽ thấy rằng số lượng phép gán và so sánh đều tăng tuyến tính với N và k, do đó thuật toán đạt O(N + k) có thể thấy ở dưới đây:

```
for N in [10*i for i in range(1, 1001)]:
    A = createRandomArray(N, k)
    a, b = create_ab(k)
    result, assignment, comparison = algorithm(A, a, b, k)
    bigO = N + k

    bigO_list.append(bigO)
    assignments.append(assignment)
    comparisons.append(comparison)
plt.figure(figsize=(10, 5))
plt.plot(bigO_list, label='O(N+k)')
plt.plot(assignments, label='Gan(N, k)')
plt.plot(comparisons, label='Sosanh(N, k)')
plt.xlabel('N')
plt.legend()
plt.show()
```

Note các hàm ngoài:

- `create_ab(k)` : Hàm này tạo ra hai số nguyên ngẫu nhiên `a` và `b` trong khoảng từ 1 đến k, đảm bảo rằng `a < b` .

```
def create_ab(k):
    a = randint(1, k)
    b = randint(1, k)
    while a >= b:
        a = randint(1, k)
        b = randint(1, k)
    return a, b
```

- `createRandomArray(N, k)` : Hàm này tạo ra một mảng ngẫu nhiên gồm N phần tử, mỗi phần tử có giá trị trong khoảng từ 1 đến k.

```
def createRandomArray(N, k):
    return [randint(1, k) for _ in range(N)]
```